02.2 Clasificación naive-Bayes de fronteras intron-exón o exón-intrón de un conjunto de secuencias de DNA

Print to PDF

La **transcripción** del ADN es el proceso en el que se transfiere la información genética contenida en el ADN, mediante la síntesis de otra molécula llamada ARN. Este proceso lo realizan las enzimas **ARN polimerasas** y tiene 3 etapas:

- **Iniciación**: la ARN polimerasa se une a una secuencia de ADN (promotor) , que se encuentra al inicio de un gen (cada gen tiene su propio promotor) y se genera un cebador de ARN al que se le van a unir los nucleótidos complementarios de la cadena molde de ADN.
- **Elongación**: la ARN polimerasa 'lee' base a base el molde de ADN y va uniendo secuencialmente, a partir del cebador, los nucleótidos de ARN complementarios a la secuencia molde formando una cadena que crece en dirección $5'->3'$. Esa cadena de ARNm (ARN mensajero), se llama transcrito y tiene la misma información, pero complementaria, que la cadena de ADN molde y en lugar de la base timina (T) tiene Uracilo (U).
- **Terminación**: una vez que la ARN polimerasa llega a las secuencias de terminación del ADN, la ARN polimerasa libera el transcrito.

Después de estas tres fases, en eucariotas, el transcrito se modifica. Primero se le añade una caperuza al extremo 5'(CAP 5') y una cola de adeninas (poli-A) al extremo 3'. También, se produce la escisión (eliminación) de los intrones (secuencias no codificantes) y la unión de los exones (secuencias codificantes) en un proceso llamado **splicing**.

El **ARNm** maduro luego va al citoplasma y se produce su **traducción a polipéptido**. En el proceso de traducción se genera una cadena polipeptídica a partir del transcrito maduro utilizando el código genético como referencia y con la ayuda del **ARNt** (ARN transferente) y las dos subunidades del ribosoma. Al igual que la transcripción, la traducción tiene tres etapas, iniciación, elongación y terminación:

- **Iniciación**: el ribosoma y el ARNt se unen al ARNm y forman el complejo de iniciación.
- **Elongación**: proceso entre ARNt y ARNm para formar los sucesivos enlaces peptídicos desde el sitio A (codón) del ribosoma.
- **Terminación**: entra al codón del ribosoma un codón de stop del ARNm, que no es reconocido por el ARNt y se produce la liberación del polipéptido y el desacoplamiento de las subunidades del ribosoma.

Se acepta como definición de **gen** a todo segmento de ADN que se encuentra a continuación de un promotor y que puede ser transcrito, originando un ARN funcional (ARNm, ARNt, ARNsn, ...).

La **transcriptoma** estudia los transcritos que se forma en el proceso de copia de la información genética contenida en el ADN en forma de ARN (**transcripción**). Actualmente existen tecnologías de secuenciación masiva llamadas **NGS** (Next Generation Sequencing). La tecnología NGS más popular es la secuenciación directa de ARN (**ARN-seq**).

La metodología **ARN-seq** tiene las etapas:

- Extracción del ARN total desde la muestra biológica.
- Fragmentación del ARN.
- Generación de librerías ARN-seq.
- Amplificación del ADN circular (por *reacción en cadena de la polimerasa*) y secuenciación para obtener pequeñas secuencias de nucleótidos (**reads**).

Consorcios científicos proporcionan gran cantidad datos de transcriptoma, genoma y proteoma:

- Proyecto **ENCODE** (ENCyclopedia Of DNA Elements).
- Proyecto **GTEX** (genotype-tissue expression).

Para datos de expresión génica, el clustering se puede realizar en función de los genes o de las muestras. En la agrupación basada en genes (*gene-based clustering*), los genes se consideran objetos mientras que las muestras se consideran características, mientras que en la agrupación basada en muestras, las muestras se pueden segregar en grupos idénticos donde los genes se tratan como características y las muestras como objetos.

"Corta-pega" en RNA:

- En algunos genes, las secciones del DNA que codifican proteínas (**exones**) a veces están interrumpidas por regiones no codificantes (**intrones**).

- El proceso de “corta-pega” del RNA **quita los intrones** del mRNA para generar la secuencia final que codifica la proteina, dejando únicamente los exones en el RNA editado. Los intrones vienen caracterizados por un sitio **donor** (frontera exón-intrón) y uno **aceptor** (frontera intrón-exón).

Una animación del proceso se muestra en: https://dnalc.cshl.edu/resources/3d/rna-splicing.html

Problema

¿**a partir de una lista de secuencias de DNA de las que conocemos “sitios donores”, ¿podemos predecir los sitios donores en nuevas secuencias**?

Conjunto de datos de entrenamiento

El conjunto de datos esta en el archivo ‘splice.txt’ que puedes descargar de Moodle. Las primeras líneas del archivo son (hemos truncado las ultimas posiciones de las secuencias solo para ajustarlas al texto de este documento):

Todas las secuencias tienen 60 pares de bases. La primera columna (**Class**) indica la clase a la que pertenece la secuencia: EI para las que proporcionan un salto exon-intrón (donores); IE para las secuencias que contienen una frontera intron-exón (aceptores); N para aquellas secuencias que no son ni EI ni IE.

La segunda columna (**Instance**) es una etiqueta para cada una de las instancias (filas), y la tercera columna da la **secuencia** correspondiente en terminos del alfabeto A, T, C y G.

Preproceso de las secuencias

Algunas de las secuencias tienen ambiguedad en determinadas posiciones, de forma que incluyen las letras D, N, S o R de acuerdo con la siguiente tabla:

Letra	Significado
D	A o G o T
N	A o G o T o C
S	C o G
R	A o G

Estos caracteres ambiguos aparecen en muy pocas instancias, así que se van a eliminar de los datos para simplificar el analisis. Primero se realiza un proceso de filtrado para saber cuantas secuencias quedan:

Finalmente se **borran** los registros de secuencia ambigua

Nota: Hay que recordar que en una lista de datos con con *true* se copian y ~(Alt-126 o insertar símbolo) niega.

Obtener dos subconjuntos, uno de entrenamiento y otro de validación

Divide el conjunto de datos en una parte para entrenamiento y otra parte para validacion, empleando muestreo aleatorio sin repetición. Reserva un 70 % de las instancias para entrenamiento y un 30 % para el conjunto de validacion

Se calculan las probabilidades de cada clase en función de la posición

Sobre el conjunto de entrenamiento, **X_train** e **y_train**.

Crear una lista de nombre, por ejemplo, **frecuencias** de tres dimensiones por clase, letra, posición. Hacer un recuento de las letras por clase y posición y guardar el resultado en la variable frecuencias. Finalmente dividir el recuento por el número de registros que tiene cada clase para obtener la frecuencia.

Sobre una lista de nombre, por ejemplo, **probaClase** guardar la frecuencia de cada clase.

Se representa la probabilidad clase a clase de cada letra en su posición

Elegir 4 colores para cada una de las letras. Hacer un plot múltiple, con 3 subplots por cada una de las clase. Por ejemplo:

fig, axs = plt.subplots(numClases, figsize=(15,10))

Crear una lista con valores de 0 a 59 para construir los valores X de cada subplot y en Y poner las frecuencias para cada una de las clases. Pasar a scatter en cada subplot un bloque completo por letra.

Se evalua el conjunto de Pruebas (test) con las frecuencias anteriores

Ir leyendo fila a fila la matriz X_test y calcular la verosimilitud en cada clase c, para las letras l y posiciones i; en base al vector de probabilidades de clase p y la matriz de frecuencias f:

$$p_c \cdot \prod_i f_{c,l,i}$$

Implementación con la librería sk-learn

Primero se prepara la matriz X de los conjuntos de entranamiento y pruebas

Para hacer el entrenamiento hay que **convertir la cadena de 60 caracteres en 60 caracteristicas de 1 caracter**:

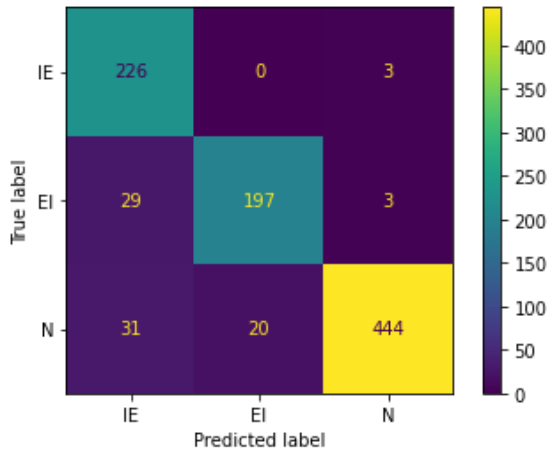
Además se convierten los caracteres alfabéticos en números

Hay varias posibilidades para hacer ésto, damos una:

Se Resuelve utilizando el clasificador Naive Bayes de sklearn

Y se presenta la correspondiente matriz de confusión

```
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_true=y_test, y_pred=y_pred)
cm_display = ConfusionMatrixDisplay(cm, display_labels=
['IE','EI','N']).plot()
```



Previous
02.1 Clasificación naive-Bayes

Next
02.3 Ratios e indicadores que califican una clasificación

By Departamento de Matemática Aplicada
© Copyright 2021.